3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

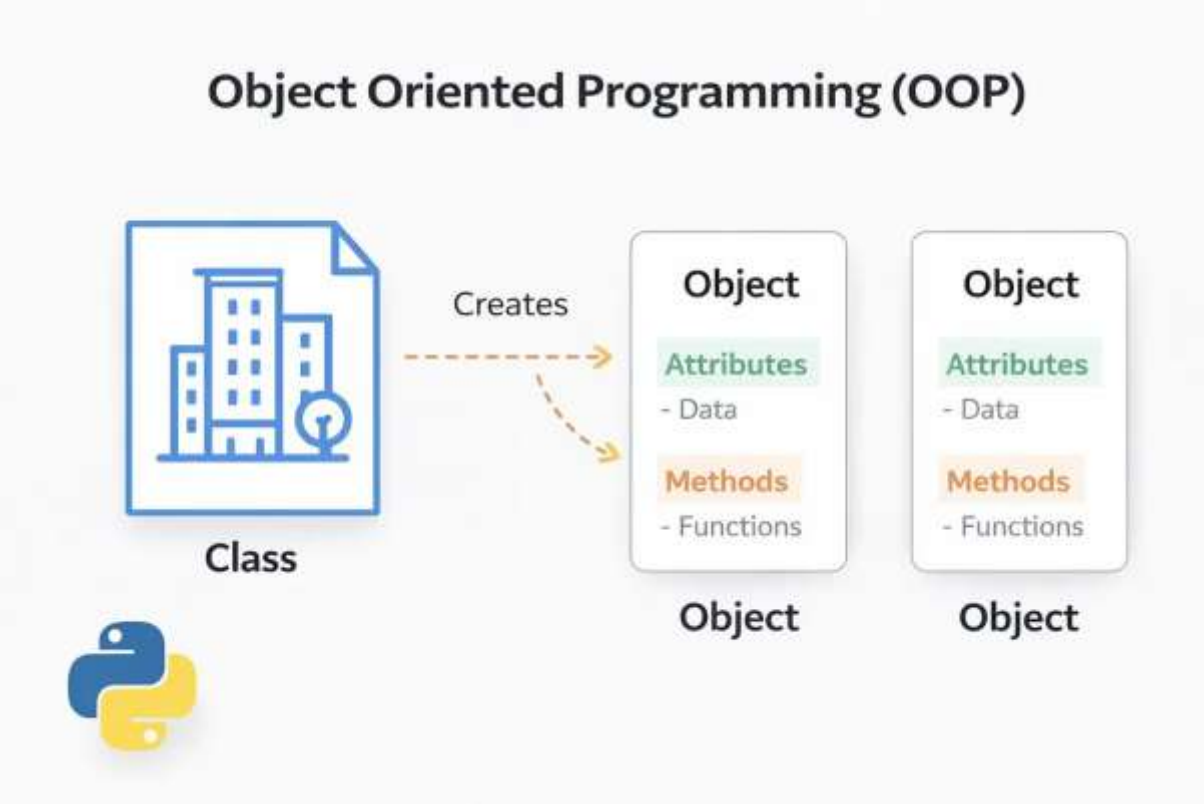
Classes : Offline & Online

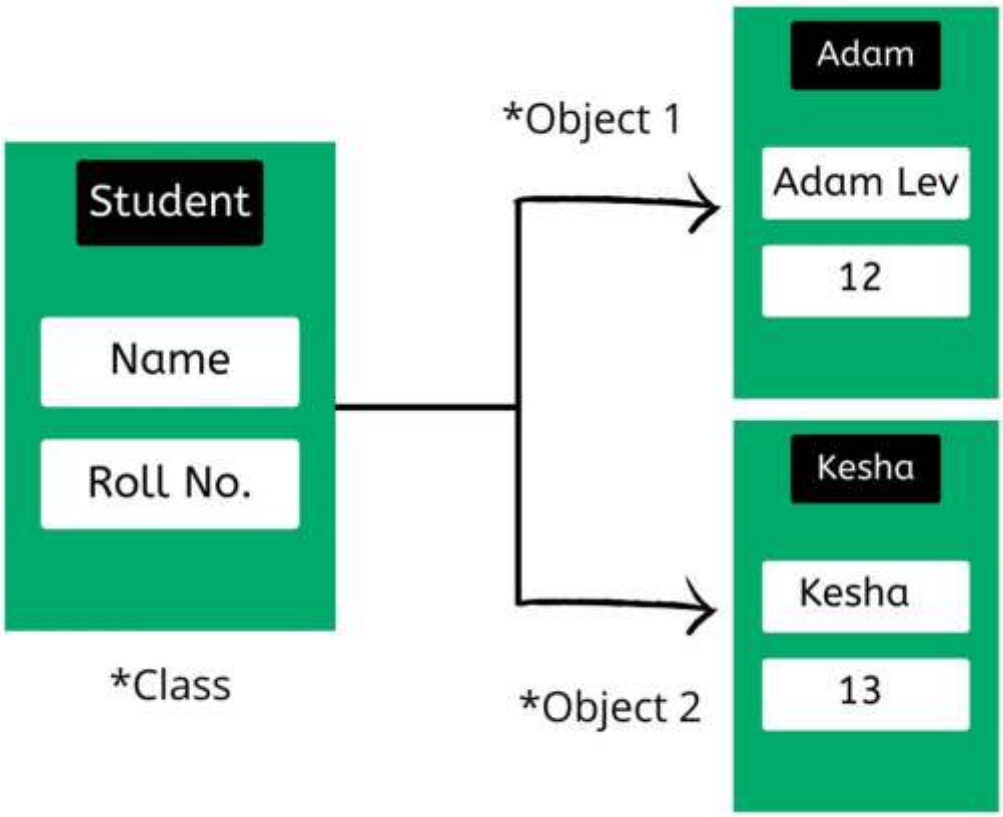
Daily Teaching : 2 Hours

Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 19 – Object-Oriented Programming (OOP)







1. Learning Objectives

By the end of this lesson, students will be able to:

- Understand the meaning of **Object-Oriented Programming**.
- Understand **classes and objects**.
- Identify attributes and methods.
- Create a class in Python.
- Create objects from a class.
- Access object attributes.
- Call object methods.
- Understand the self keyword.
- Use the __init__() constructor.
- Build practical applications using classes and objects.

2. What is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming approach where programs are designed using **objects**.

An object represents something that has:

- **Data** → what it has
- **Behavior** → what it can do

Real-world example

Think about a **Student**.

A student has:

Data:

- Name
- Roll number

3DWEBSOFTFOUNDATION

- Age
- Marks

Behavior:

- Study
- Attend class
- Display details
- Calculate result

We can represent this concept using a Python object.

```
          STUDENT OBJECT
               |
       ----------------------
       |                |
     DATA             BEHAVIOR
       |                |
  Name, Roll, Age     Study()
  Marks, Course       Display()
                      Result()
```

3. Why Do We Need OOP?

Suppose we have three students.

Without OOP:

```
student1_name = "Rahul"
student1_age = 17
```

3DWEBSOFT FOUNDATION

student1_marks = 85

student2_name = "Priya"

student2_age = 16

student2_marks = 92

As the number of students increases, the program becomes difficult to manage.

With OOP:

student1 = Student("Rahul", 17, 85)

student2 = Student("Priya", 16, 92)

The program becomes more organized.

OOP helps us:

- Organize large programs.
- Reuse code.
- Represent real-world entities.
- Keep related data and functions together.
- Make programs easier to maintain.

4. What is a Class?

A **class** is a blueprint or template for creating objects.

Think about a house blueprint.

CLASS

House Blueprint

|

3DWEBSOFT FOUNDATION

| |

Object 1 Object 2

House House

The blueprint describes what a house can contain, but the blueprint itself is not a physical house.

Similarly:

Class = Blueprint
Object = Actual instance created from the blueprint

5. What is an Object?

An **object** is an instance of a class.

Example:

```
class Student:
    pass
```

Here, Student is a class.

We can create an object:

```
student1 = Student()
```

Here:

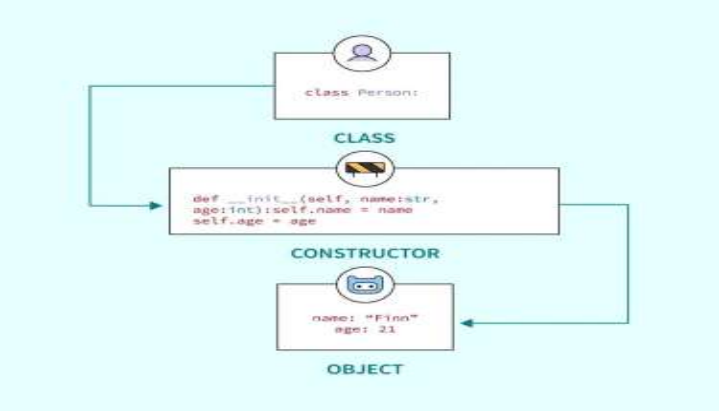
Student → Class

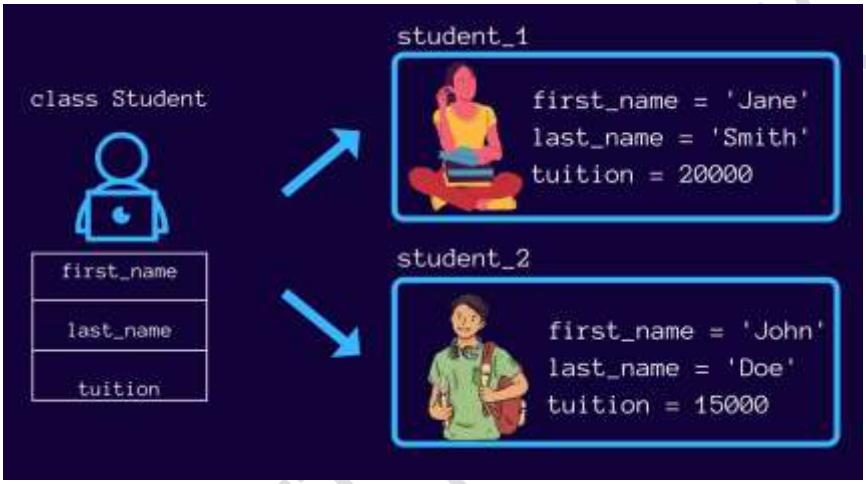
student1 → Object

6. Class and Object Relationship

3DWEBSOFT FOUNDATION







CLASS

Student

|

| | |

↓ ↓ ↓

Object 1 Object 2 Object 3

Rahul Priya Arjun

One class can create many objects.

3DWEBSOFT

7. Creating a Simple Class

Syntax:

```
class ClassName:
    # attributes
    # methods
```

Example:

```
class Student:
    pass
```

pass means:

"Do nothing for now."

8. Creating an Object

```
class Student:
    pass
student1 = Student()
student2 = Student()
print(student1)
print(student2)
```

Here:

Student

↓

| |

SUWEBSOFT FOUNDATION

student1 student2

Both are separate objects created from the same class.

9. Attributes

Attributes are the data or properties associated with an object.

For a student:

Name

Age

Roll Number

Marks

Course

Example:

```
class Student:
    pass


student1 = Student()
student1.name = "Rahul"
student1.age = 17
student1.marks = 85
print(student1.name)
print(student1.age)
print(student1.marks)
```

Output

SUWEBSOFT FOUNDATION

Rahul

17

85

10. Methods

A **method** is a function defined inside a class.

Example:

```
class Student:
    def study(self):
        print("Student is studying.")
```

Create an object:

```
student1 = Student()
student1.study()
```

Output:

Student is studying.

Remember

Function outside class → Function

Function inside class → Method

11. The self Keyword

One of the most important concepts in Python OOP is:

self

self refers to the **current object**.

3DWEBSOFT FOUNDATION

Example:

```
class Student:
    def display(self):
        print("Student details")
```

When we call:

```
student1.display()
```

Python uses student1 as the current object.

Simple understanding

```
student1.display()
        |
        ↓
      self
        |
        ↓
    student1
```

12. Creating a Student Class

Let's create a useful class.

```
class Student:
    def display(self):
        print("Student Name: Rahul")
        print("Course: Python")
student1 = Student()
```

3DWEBSOFT FOUNDATION

student1.display()

Output:

Student Name: Rahul

Course: Python

But this class always displays Rahul.

We need a better design.

13. The __init__() Method

The __init__() method is commonly used to initialize an object's attributes when the object is created.

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Now create an object:

```
student1 = Student("Rahul", 17)
print(student1.name)
print(student1.age)
```

Output:

Rahul

17

3DWEBSOFT FOUNDATION

14. Understanding __init__()

When we write:

student1 = Student("Rahul", 17)

Python creates the object and initializes its data using:

__init__()

Flow

Student("Rahul", 17)

↓

__init__()

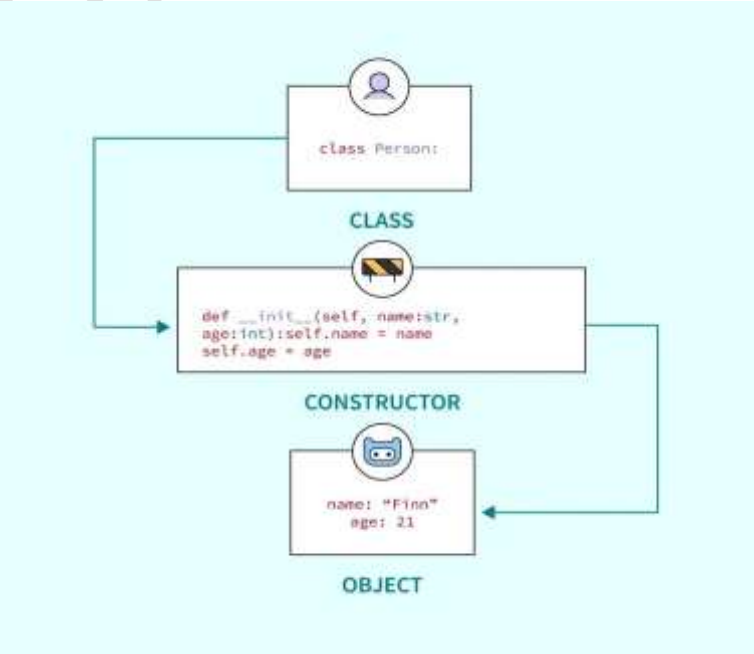
↓

self.name = "Rahul"

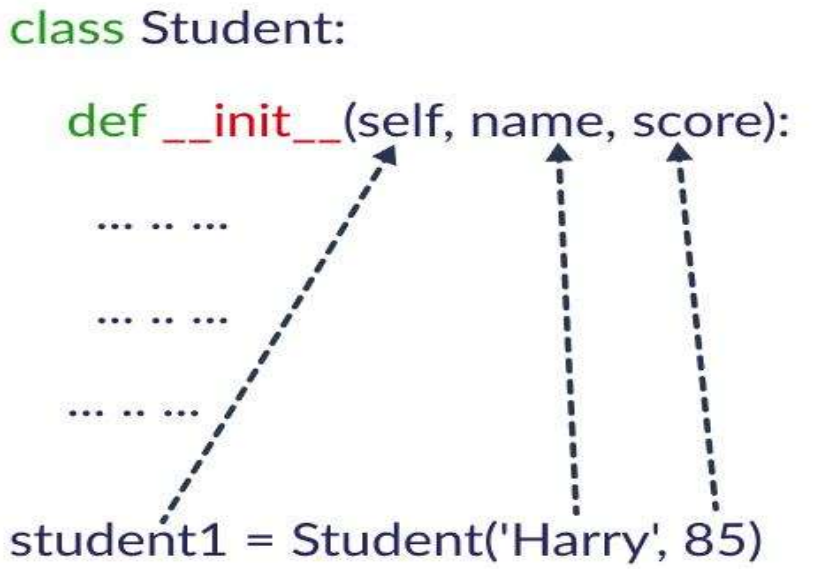
self.age = 17

↓

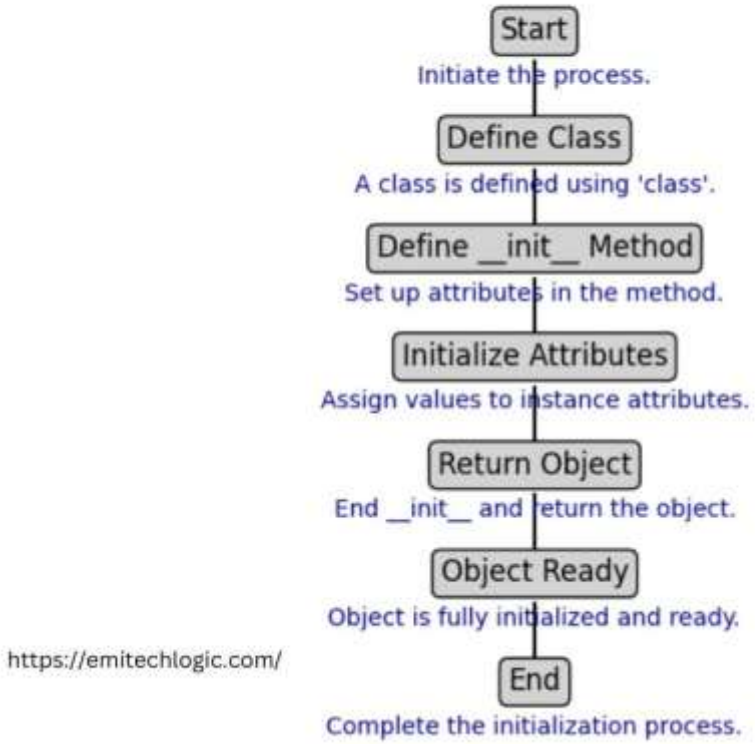
student1 object



3DWEBSOFT FOUNDATION



Concept of the __init__ Method in Python



15. Complete Student Class

class Student:

```
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks
    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.marks)
student1 = Student("Rahul", 17, 85)
student1.display()
```

Output

Name: Rahul

Age: 17

Marks: 85

16. Creating Multiple Objects

The real power of classes is that one class can create many objects.

```
class Student:
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks
    def display(self):
```

3DWEBSOFT FOUNDATION

```
        print(self.name, self.age, self.marks)
student1 = Student("Rahul", 17, 85)
student2 = Student("Priya", 16, 92)
student3 = Student("Arjun", 17, 78)
student1.display()
student2.display()
student3.display()
```

Output

```
Rahul 17 85
Priya 16 92
Arjun 17 78
```

17. Object Data is Independent

Each object has its own data.

```
student1 = Student("Rahul", 17, 85)
student2 = Student("Priya", 16, 92)
```

Think:

student1

Name → Rahul

Age → 17

Marks → 85

student2

Name → Priya

3DWEBSOFT FOUNDATION

Age → 16

Marks → 92

Changing one object does not automatically change the other.

18. Adding Methods to a Class

A class can contain many methods.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)
    def result(self):
        if self.marks >= 40:
            print("Result: Pass")
        else:
            print("Result: Fail")
```

Use it:

```
student1 = Student("Rahul", 75)
student1.display()
student1.result()
```

Output:

3DWEBSOFT FOUNDATION

Name: Rahul

Marks: 75

Result: Pass

19. Practical – Student Result System

```
class Student:
    def __init__(self, name, roll, marks):
        self.name = name
        self.roll = roll
        self.marks = marks
    def display(self):
        print("\n--- Student Details ---")
        print("Name:", self.name)
        print("Roll:", self.roll)
        print("Marks:", self.marks)
    def result(self):
        if self.marks >= 40:
            print("Result: PASS")
        else:
            print("Result: FAIL")
student1 = Student("Rahul", 101, 85)
student1.display()
student1.result()
```

3DWEBSOFT FOUNDATION

Output

--- Student Details ---

Name: Rahul

Roll: 101

Marks: 85

Result: PASS

20. Practical – Employee Class

Now create an Employee class.

```
class Employee:
    def __init__(self, name, department, salary):
        self.name = name
        self.department = department
        self.salary = salary
    def display(self):
        print("Name:", self.name)
        print("Department:", self.department)
        print("Salary:", self.salary)
employee1 = Employee("Aisha", "IT", 35000)
employee1.display()
```

Output:

Name: Aisha

Department: IT

3DWEBSOFT FOUNDATION

Salary: 35000

21. Practical – Bank Account

OOP is useful for modeling bank accounts.

```
class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print("Withdrawal successful.")
        else:
            print("Insufficient balance.")
    def display_balance(self):
        print("Account Holder:", self.account_holder)
        print("Balance:", self.balance)
account1 = BankAccount("Rahul", 5000)
account1.deposit(2000)
account1.withdraw(1000)
account1.display_balance()
```

3DWEBSOFT FOUNDATION

Output

Withdrawal successful.

Account Holder: Rahul

Balance: 6000

22. Practical – Calculator Class

```
class Calculator:
    def add(self, a, b):
        return a + b
    def subtract(self, a, b):
        return a - b
    def multiply(self, a, b):
        return a * b
    def divide(self, a, b):
        if b == 0:
            return "Cannot divide by zero."
        return a / b
calc = Calculator()
print("Addition:", calc.add(10, 5))
print("Subtraction:", calc.subtract(10, 5))
print("Multiplication:", calc.multiply(10, 5))
print("Division:", calc.divide(10, 5))
```

Output

SUWEBSOFT FOUNDATION

Addition: 15

Subtraction: 5

Multiplication: 50

Division: 2.0

23. Class Attributes vs Instance Attributes

This is an important concept for 11th–12th standard students.

Instance attribute

Belongs to a particular object.

self.name

self.age

self.marks

Class attribute

Shared by objects of the class unless overridden.

```
class Student:
    school = "ABC School"
    def __init__(self, name):
        self.name = name
```

Now:

```
student1 = Student("Rahul")
student2 = Student("Priya")
print(student1.school)
print(student2.school)
```

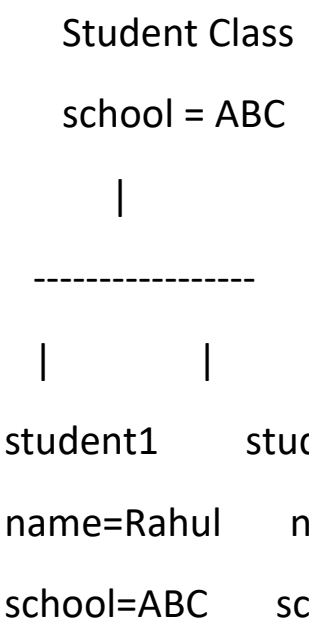
3DWEBSOFT FOUNDATION

Output:

ABC School

ABC School

Diagram



24. Changing an Object's Attribute

Attributes can be changed after object creation.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
student1 = Student("Rahul", 70)
print(student1.marks)
student1.marks = 85
print(student1.marks)
```

3DWEBSOFT FOUNDATION

Output:

70

85

25. Adding a New Attribute

Python also allows us to add an attribute to an object.

```
class Student:
    pass

student1 = Student()

student1.name = "Rahul"

student1.city = "Hyderabad"

print(student1.name)

print(student1.city)
```

However, for well-structured applications, it is usually better to define expected attributes clearly in __init__().

26. OOP Real-World Examples

Real World	**Python Class**
Student	Student
Teacher	Teacher
Bank account	BankAccount
Car	Car

3DWEBSOFT FOUNDATION

Real World	Python Class
Employee	Employee
Product	Product
Book	Book
Hospital patient	Patient
Library book	LibraryBook

Example

Class: Car

Attributes:

brand

model

color

speed

Methods:

start()

stop()

accelerate()

brake()

27. Practical – Car Class

3DWEBSOFT FOUNDATION

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model
    def start(self):
        print(self.brand, "is starting.")
    def stop(self):
        print(self.brand, "has stopped.")
car1 = Car("Toyota", "Camry")
car1.start()
car1.stop()
```

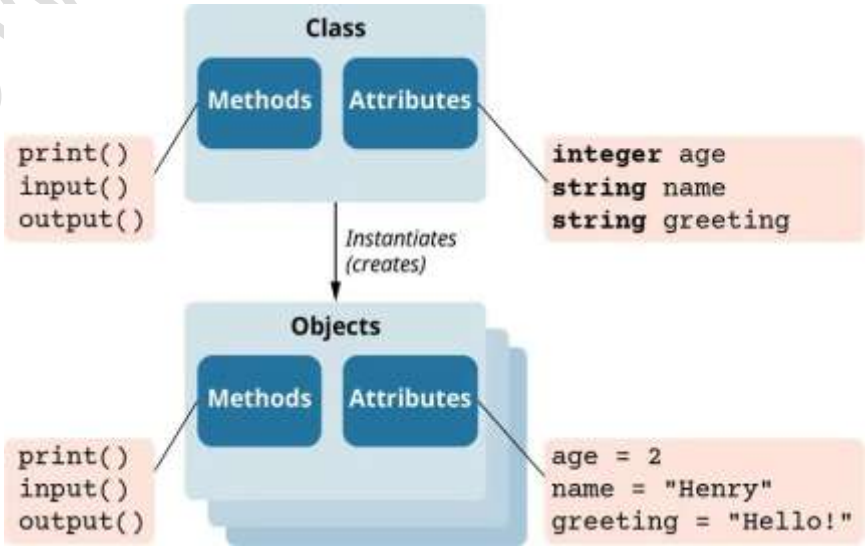
Output:

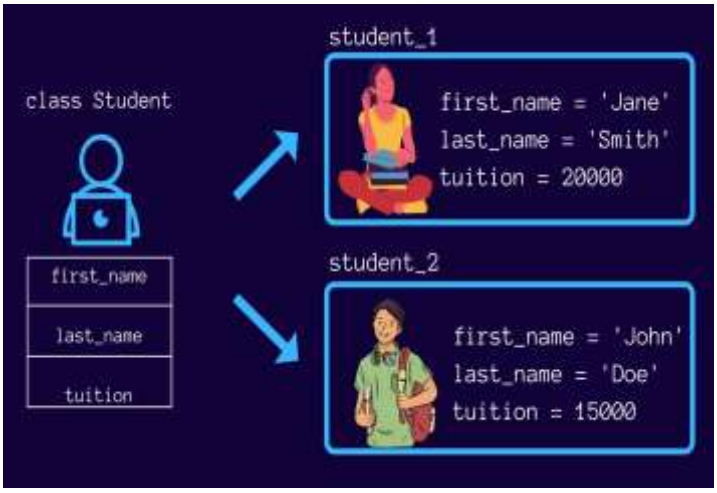
Toyota is starting.

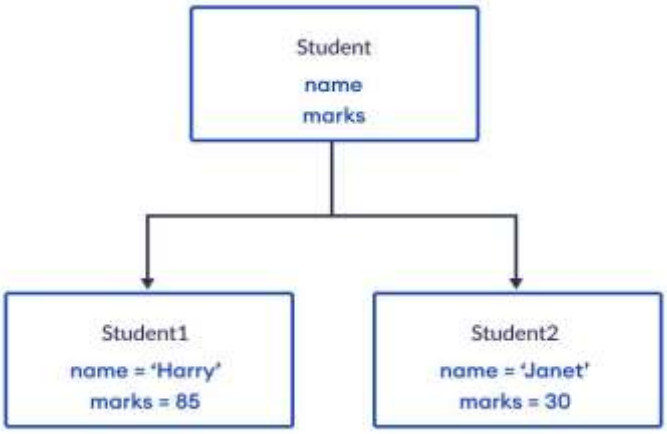
Toyota has stopped.

3DWEBSOFT FOUNDATION

28. Class and Object Diagram







CLASS

Student

|

| |

Attributes Methods

| |

name display()

age result()

marks study()

|

↓

OBJECT

student1

|

Rahul / 17 / 85

3DWEBSOFT FOUNDATIO

29. OOP Program Structure

A typical beginner OOP program looks like:

```
class ClassName:


    def __init__(self, data):
        self.data = data


    def method(self):
        # action
        pass
object1 = ClassName(value)
object1.method()
```

Students should understand this structure before moving to advanced OOP concepts.

30. OOP vs Procedural Programming

Procedural Programming	**OOP**
Focuses on functions/procedures	Focuses on objects
Data and functions can be separate	Data and methods are grouped
Suitable for small programs	Very useful for larger programs

3DWEBSOFT FOUNDATION

Procedural Programming	OOP
Reuse mainly through functions	Reuse through classes/objects and other OOP features
Example: collection of functions	Example: Student objects

Simple idea

Procedural:

Data → Functions

OOP:

Object → Data + Methods

31. Practical – Product Management

```
class Product:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity
    def total_price(self):
        return self.price * self.quantity
    def display(self):
        print("Product:", self.name)
        print("Price:", self.price)
```

3DWEBSOFT FOUNDATION

```
        print("Quantity:", self.quantity)
        print("Total:", self.total_price())
product1 = Product("Laptop", 50000, 2)
product1.display()
```

Output

```
Product: Laptop
Price: 50000
Quantity: 2
Total: 100000
```

32. Practical Activity – Design a Class

Ask students to choose one real-world object.

Option 1 – Mobile Phone

Attributes:

brand

model

price

storage

Methods:

call()

message()

display()

Option 2 – Book

3DWEBSOFT FOUNDATION

Attributes:

title

author

price

pages

Methods:

display()

read()

Option 3 – Student

Attributes:

name

roll

age

marks

Methods:

display()

result()

3DWEBSOFT FOUNDATION

33. Mini Project – Student Management Using OOP

Project Name

OOP Student Management System

The application should use a Student class.

Student Class

```
                |
    ------------------------------
    |           |           |
Attributes    Methods     Objects
    |           |           |
name          display()   student1
roll          result()    student2
age           grade()     student3
marks
```

Basic implementation

```
class Student:
    def __init__(self, name, roll, age, marks):
        self.name = name
        self.roll = roll
        self.age = age
        self.marks = marks
    def display(self):
        print("\nName:", self.name)
        print("Roll:", self.roll)
        print("Age:", self.age)
        print("Marks:", self.marks)
    def result(self):
        if self.marks >= 40:
```

3DWEBSOFT FOUNDATION

```
            return "Pass"
        return "Fail"
    def grade(self):
        if self.marks >= 90:
            return "A"
        elif self.marks >= 75:
            return "B"
        elif self.marks >= 60:
            return "C"
        elif self.marks >= 40:
            return "D"
        else:
            return "F"
student1 = Student("Rahul", 101, 17, 85)
student1.display()
print("Result:", student1.result())
print("Grade:", student1.grade())
```

Output

```
Name: Rahul
Roll: 101
Age: 17
Marks: 85
Result: Pass
```

3DWEBSOFT FOUNDATION

Grade: B

34. Connecting OOP with Previous Lessons

Students have already learned:

Class 10 → *args and **kwargs

Class 11 → Lambda Functions

Class 12 → map(), filter(), reduce()

Class 13 → Modules & Packages

Class 14 → File Handling

Class 15 → CSV

Class 16 → JSON

Class 17 → Exception Handling

Class 18 → Debugging

Now OOP provides a way to **organize all these concepts into reusable application structures**.

For example:

Student Class

|

+— JSON data

|

+— File handling

|

+— Exception handling

3DWEBSOFT FOUNDATION

|

+— Functions/methods

|

+— Data processing

This is an important step toward building larger Python applications.

35. Common Mistakes

Mistake 1 – Forgetting self

Incorrect:

```
class Student:
    def display():
        print("Student")
```

Correct:

```
class Student:
    def display(self):
        print("Student")
```

Mistake 2 – Incorrect attribute assignment

Incorrect:

```
def __init__(self, name):
    name = name
```

Correct:

3DWEBSOFT FOUNDATION

```
def __init__(self, name):
    self.name = name
```

Mistake 3 – Calling a method incorrectly

If:

```
student1.display()
```

is the method call, don't forget the parentheses.

Mistake 4 – Wrong class name

```
class Student:
    pass
student1 = Students()
```

Students and Student are different names.

36. Debugging OOP Programs

When debugging a class, check:

1. Is the class name correct?
2. Is __init__ written correctly?
3. Is self included?
4. Are attributes written as self.attribute?
5. Are method names correct?
6. Are the correct arguments passed?
7. Is the object created correctly?

3DWEBSOFT FOUNDATION

8. Are methods called correctly?

Example:

student1 = Student("Rahul", 17, 85)

Check that the constructor expects:

def __init__(self, name, age, marks):

37. Classroom Exercise

Task 1

Create a Book class with:

title

author

price

and a method:

display()

Task 2

Create two book objects.

Task 3

Display both books.

Task 4

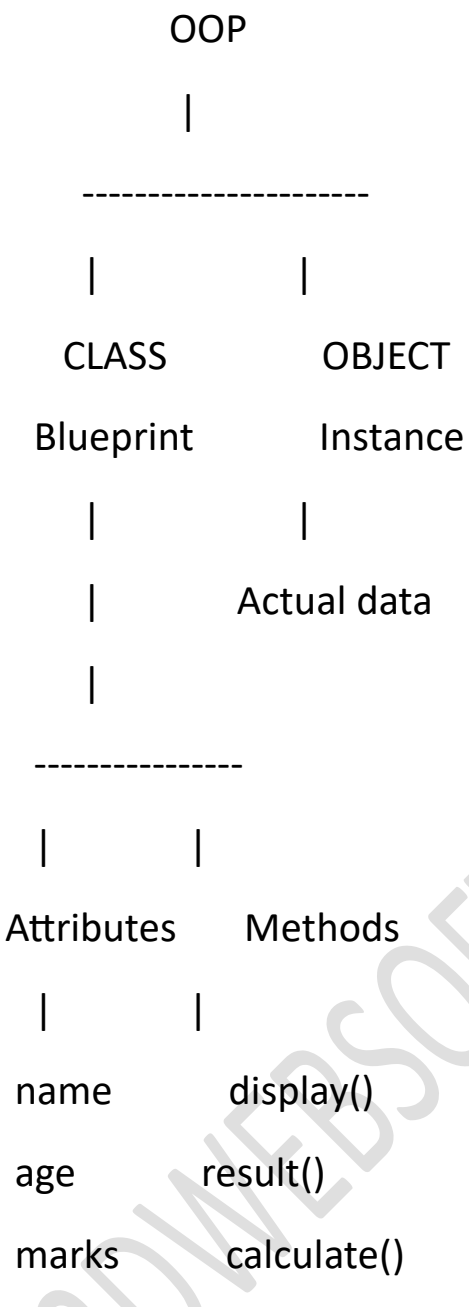
Add a method:

discount()

that calculates a 10% discount.

3DWEBSOFT FOUNDATION

38. Quick Revision



Important terms

Term	**Meaning**
Class	Blueprint/template
Object	Instance of a class
Attribute	Object's data/property

3DWEBSOFT FOUNDATION

Term	Meaning
Method	Function inside a class
self	Reference to current object
__init__()	Initialization method
Instance	Individual object created from a class

39. Knowledge Check – MCQs

1. What is a class?

A. A loop
B. A blueprint for objects
C. A variable
D. An error

Answer: B

2. What is an object?

A. An instance of a class
B. A Python keyword
C. A loop
D. A module

Answer: A

3. Which method is commonly used to initialize an object?

A. start()
B. main()
C. __init__()
D. create()

3DWEBSOFT FOUNDATION

Answer: C

4. What does self refer to?

A. Parent class
B. Current object
C. Python interpreter
D. Module

Answer: B

5. A function inside a class is called:

A. Attribute
B. Object
C. Method
D. Variable

Answer: C

6. Which statement creates an object?

```
class Student:
    pass
```

A. Student.create()
B. student1 = Student()
C. object Student()
D. new Student()

Answer: B

3DWEBSOFT FOUNDATION

40. Practical Assignment

Create a BankAccount Class

Students must create a class containing:

Attributes

account_holder

account_number

balance

Methods

deposit()

withdraw()

display_balance()

Requirements

- Deposit money.
- Prevent withdrawal greater than balance.
- Display current balance.
- Create at least two account objects.
- Test different transactions.

Bonus

Add exception handling:

```
try:
    amount = float(input("Enter amount: "))
except ValueError:
    print("Please enter a valid amount.")
```

41. Final Takeaway

The most important concepts from Class 19 are:

3DWEBSOFT FOUNDATION

CLASS

↓

Blueprint

↓

OBJECT

↓

Data + Behavior

↓

Attributes + Methods

↓

__init__()

↓

Initialize Object

↓

self

↓

Current Object

3DWEBSOFT FOUNDATION